

Universidad Nacional Autónoma de México
Facultad de Ciencias
Licenciatura en Ciencias de la Computación

Seminario de Ciencias de la Computación B
Computación en la Nube

Proyecto Final

Programación del patrón Circuit Breaker con Resilience4J

Integrante	No. de Cuenta
Elizalde Maza Jesús Eduardo	321031686
Peredo López Citlalli Abigail	321161022

Profesor: Gustavo Arturo Márquez Flores
10 de junio de 2026

Objetivo

Programar el patrón **Circuit Breaker** mediante las clases de **Resilience4J** utilizando las anotaciones **@CircuitBreaker** y **@TimeLimiter**, para mostrar el comportamiento de un microservicio ante fallas y latencia de tiempo.

Especificación

Continuación de la Tarea 5. Las actividades programadas son:

- Especificar en *application.yml* los parámetros de configuración de Resilience4J.
- Programar un método alternativo por **falla** usando **@CircuitBreaker** con `throw new IllegalStateException("Producto no encontrado!")`.
- Programar un método alternativo por **latencia** usando **@TimeLimiter** con `TimeUnit.SECONDS.sleep(5L)`.
- Registrar mensajes de error en el *logger* dentro de cada método alternativo.

Arquitectura del Sistema

El sistema está compuesto por cuatro microservicios que se comunican a través de Eureka Server:

Puerto	Microservicio	Rol
8761	springboot-servicio-eureka-server	Servidor de descubrimiento (Eureka)
8001	springboot-servicio-productos	CRUD de productos con H2 y vista Thymeleaf
8002	springboot-servicio-items	Lógica de negocio, consume servicio-productos vía Feign. Implementa CircuitBreaker y TimeLimiter
8090	springboot-servicio-zuul-server	API Gateway con ruteo dinámico y FallbackProvider

Panel de Eureka — servicios registrados:

1. Configuración de Resilience4J en application.yml

Se especificaron los parámetros de Resilience4J en el archivo *application.yml* del microservicio **servicio-items**. Se definió una configuración base llamada *defecto* referenciada por dos instancias: **itemsCB** (CircuitBreaker de falla) e **itemsTL** (TimeLimiter de latencia).

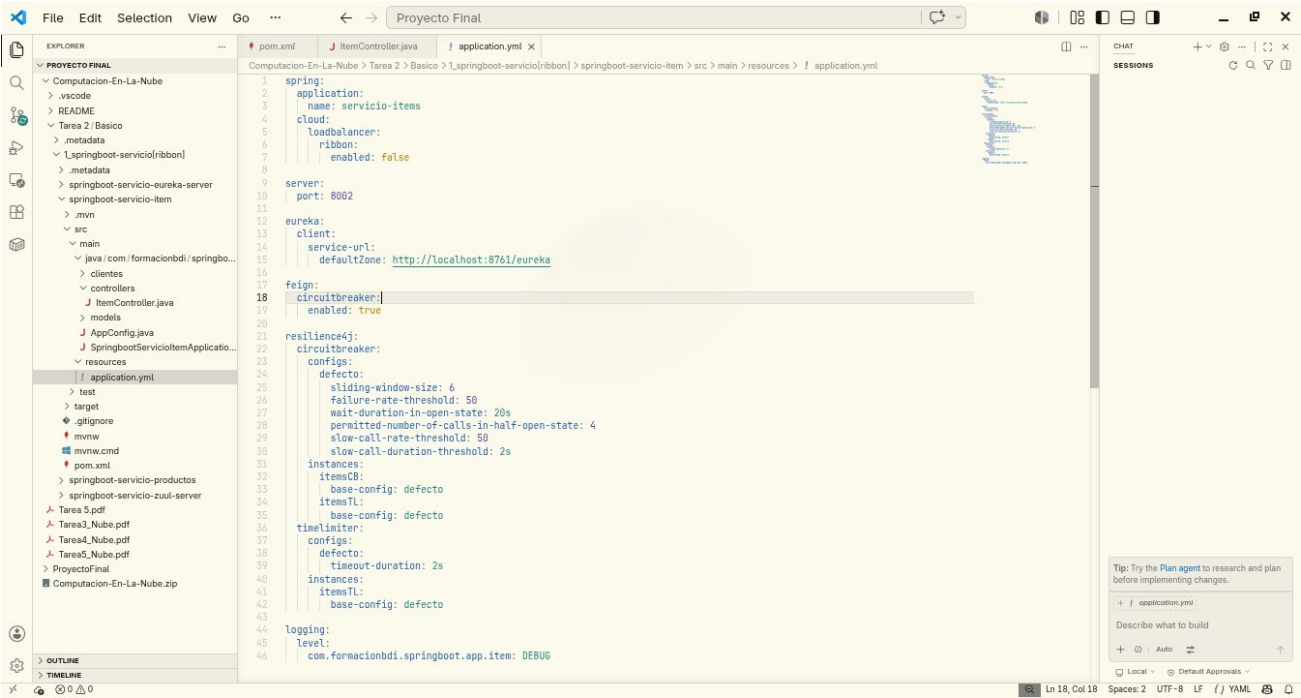


Figura 2. Archivo *application.yml* mostrando la configuración completa de Resilience4J con instancias *itemsCB* e *itemsTL*.

Significado de cada parámetro:

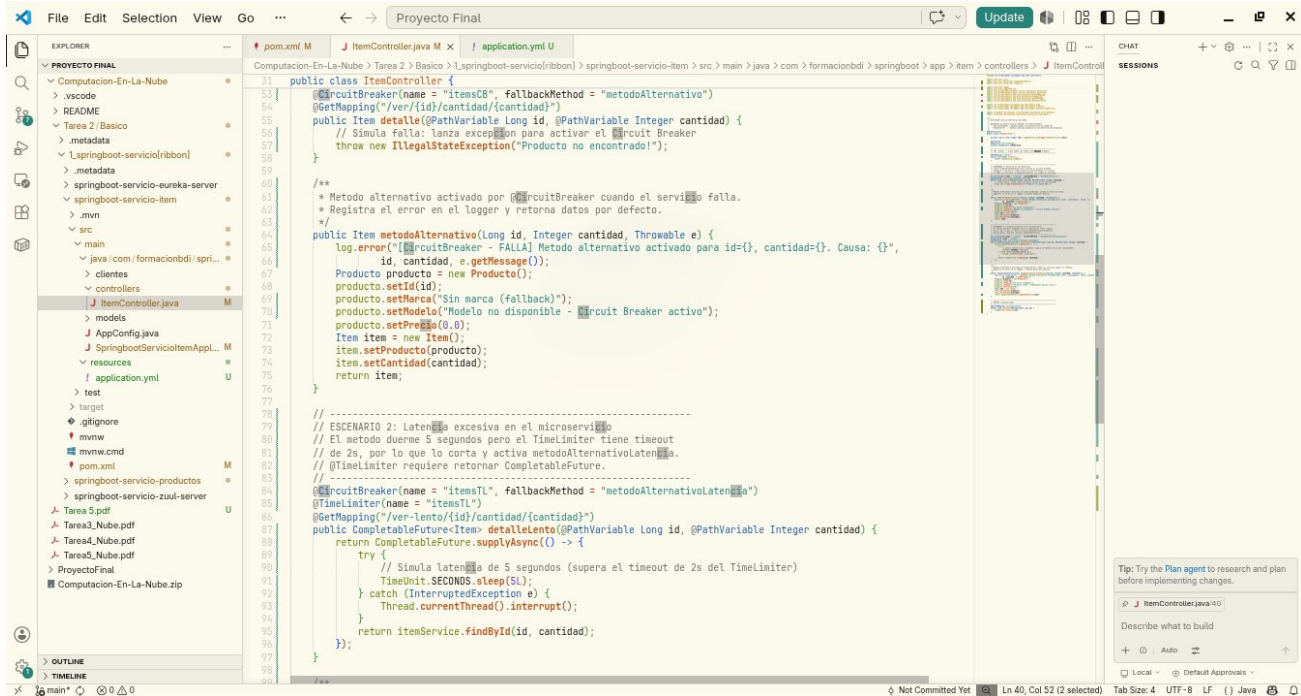
Parámetro	Valor	Descripción
sliding-window-size	6	Número de llamadas analizadas para calcular la tasa de fallos
failure-rate-threshold	50%	Si el 50% de llamadas fallan, el circuito se ABRE
wait-duration-in-open-state	20s	Tiempo que el circuito permanece ABIERTO
permitted-calls-in-half-open	4	Llamadas de prueba en estado HALF-OPEN
slow-call-rate-threshold	50%	Si 50% de llamadas son lentas, el circuito se ABRE
slow-call-duration-threshold	2s	Umbral para considerar una llamada como lenta
timeout-duration (TimeLimiter)	2s	Tiempo máximo antes de activar el método alternativo

2. Circuit Breaker por Falla — @CircuitBreaker

2.1 Descripción

Se implementó el escenario de falla en el endpoint **GET /ver/{id}/cantidad/{cantidad}** del controlador *ItemController*. El método lanza deliberadamente una *IllegalStateException* para simular que el microservicio de productos no está disponible. El Circuit Breaker intercepta la excepción y ejecuta el método alternativo **metodoAlternativo()**.

2.2 Código implementado



```
53 public class ItemController {
54     @CircuitBreaker(name = "itemsCB", fallbackMethod = "metodoAlternativo")
55     @GetMapping("/ver/{id}/cantidad/{cantidad}")
56     public Item detalle(@PathVariable Long id, @PathVariable Integer cantidad) {
57         // Simula falla: lanza excepción para activar el Circuit Breaker
58         throw new IllegalStateException("Producto no encontrado!");
59     }
60
61     /**
62      * Método alternativo activado por @CircuitBreaker cuando el servicio falla.
63      * Registra el error en el logger y retorna datos por defecto.
64      */
65     public Item metodoAlternativo(Long id, Integer cantidad, Throwable e) {
66         log.error("@CircuitBreaker - FALLA! Método alternativo activado para id={}, cantidad={}. Causa: {}",
67             id, cantidad, e.getMessage());
68         Producto producto = new Producto();
69         producto.setId(id);
70         producto.setMarca("Sin marca (fallback)");
71         producto.setModelo("Modelo no disponible - @Circuit Breaker activo");
72         producto.setPrecio(0.0);
73         Item item = new Item();
74         item.setProducto(producto);
75         item.setCantidad(cantidad);
76         return item;
77     }
78
79     // -----
80     // ESCENARIO 2: Latencia excesiva en el microservicio
81     // El método duerme 5 segundos pero el TimeLimiter tiene timeout
82     // de 2s, por lo que lo corta y activa metodoAlternativoLento.
83     // @TimeLimiter requiere retornar CompletableFuture.
84     @CircuitBreaker(name = "itemsTL", fallbackMethod = "metodoAlternativoLento")
85     @TimeLimiter(name = "itemsTL")
86     @GetMapping("/ver-lento/{id}/cantidad/{cantidad}")
87     public CompletableFuture<Item> detalleLento(@PathVariable Long id, @PathVariable Integer cantidad) {
88         return CompletableFuture.supplyAsync(() -> {
89             try {
90                 // Simula latencia de 5 segundos (supera el timeout de 2s del TimeLimiter)
91                 TimeUnit.SECONDS.sleep(5L);
92             } catch (InterruptedException e) {
93                 Thread.currentThread().interrupt();
94             }
95             return itemService.findById(id, cantidad);
96         });
97     }
98 }
```

Figura 3. *ItemController.java* — implementación de *@CircuitBreaker* (líneas 53–76) y *@TimeLimiter* (líneas 84–97) con sus métodos alternativos.

2.3 Evidencia — Respuesta JSON del fallback (CircuitBreaker)

Al llamar al endpoint **GET /ver/1/cantidad/2**, el *CircuitBreaker* detecta la *IllegalStateException* y devuelve inmediatamente los datos del método alternativo con marca *"Sin marca (fallback)"*.

1 3 4 5 martes

```
base ) curl http://localhost:8002/ver/1/cantidad/2 (base)
{"producto":{"id":1,"marca":"Sin marca (fallback)","modelo":"Modelo no disponible - Circuit
Breaker activo","anio":null,"precio":0.0,"createAt":null},"cantidad":2,"total":0.0,"factura
a":"Número de Factura 3444523","iva":0.0}

base ) curl http://localhost:8002/ver-lento/1/cantidad/1 (base)
{"producto":{"id":1,"marca":"Sin marca (timeout)","modelo":"Servicio lento - TimeLimiter ac
tivo (>2s)","anio":null,"precio":0.0,"createAt":null},"cantidad":1,"total":0.0,"factura":"N
úmero de Factura 3444523","iva":0.0}

base ) curl http://localhost:8002/listar (base)
[{"producto":{"id":1,"marca":"Ford","modelo":"Focus","anio":2010,"precio":250000.0,"createA
t":"2026-06-09T00:00:00.000+00:00"},"cantidad":1,"total":250000.0,"factura":"Número de Fact
ura 3444523","iva":37500.0}, {"producto":{"id":2,"marca":"Ford","modelo":"Fusion","anio":201
6,"precio":350000.0,"createAt":"2026-06-09T00:00:00.000+00:00"},"cantidad":1,"total":350000
.0,"factura":"Número de Factura 3444523","iva":52500.0}, {"producto":{"id":3,"marca":"GMC",
"modelo":"Acadia","anio":2026,"precio":1352000.0,"createAt":"2026-06-09T00:00:00.000+00:00"}
,"cantidad":1,"total":1352000.0,"factura":"Número de Factura 3444523","iva":202800.0}, {"pro
ducto":{"id":4,"marca":"GMC","modelo":"Acadia","anio":2026,"precio":1352900.0,"createAt":"2
```

Figura 4. Respuestas en terminal: curl al endpoint /ver/1/cantidad/2 retorna fallback 'Sin marca (fallback)' (CircuitBreaker), y /ver-lento/1/cantidad/1 retorna fallback 'Sin marca (timeout)' (TimeLimiter). El endpoint /listar muestra la lista normal de productos.

2.4 Evidencia — Mensajes en el logger

El microservicio registra en consola los errores de ambos patrones. A la derecha se observan los logs del **servicio-items** (puerto 8002) con los prefijos **[CircuitBreaker - FALLA]** y **[TimeLimiter - LATENCIA]**.

19:59



```
UP (replication=true)
2026-06-09 19:55:11.238 INFO 45781 --- [a-EvictionTimer] c.n.e.registry.AbstractInstanceRegistry : Running the evict task with compensationTime 0ms
2026-06-09 19:56:11.238 INFO 45781 --- [a-EvictionTimer] c.n.e.registry.AbstractInstanceRegistry : Running the evict task with compensationTime 0ms
2026-06-09 19:57:11.238 INFO 45781 --- [a-EvictionTimer] c.n.e.registry.AbstractInstanceRegistry : Running the evict task with compensationTime 0ms
2026-06-09 19:58:11.238 INFO 45781 --- [a-EvictionTimer] c.n.e.registry.AbstractInstanceRegistry : Running the evict task with compensationTime 0ms
2026-06-09 19:59:11.239 INFO 45781 --- [a-EvictionTimer] c.n.e.registry.AbstractInstanceRegistry : Running the evict task with compensationTime 0ms
[]
```

```
2026-06-09 19:55:08.503 INFO 49597 --- [nfoReplicator-0] com.netflix.discovery.DiscoveryClient : DiscoveryClient_SERVICIO-ITEMS/192.168.0.2:servicio-items:8002: registering service...
2026-06-09 19:55:08.520 INFO 49597 --- [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8002 (http) with context path ''
2026-06-09 19:55:08.521 INFO 49597 --- [ restartedMain] .s.c.n.e.s.EurekaAutoServiceRegistration : Updating port to 8002
2026-06-09 19:55:08.536 INFO 49597 --- [ restartedMain] .s.a.i.SpringbootServicioItemApplication : Started SpringbootServicioItemApplication in 6.142 seconds (JVM running for 6.774)
2026-06-09 19:55:08.549 INFO 49597 --- [nfoReplicator-0] com.netflix.discovery.DiscoveryClient : DiscoveryClient_SERVICIO-ITEMS/192.168.0.2:servicio-items:8002 - registration status: 204
2026-06-09 19:58:54.022 INFO 49597 --- [nio-8002-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-06-09 19:58:54.022 INFO 49597 --- [nio-8002-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-06-09 19:58:54.025 INFO 49597 --- [nio-8002-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 3 ms
2026-06-09 19:58:54.144 ERROR 49597 --- [nio-8002-exec-1] c.f.s.a.item.controllers.ItemController : [CircuitBreaker - FALLA] Metodo alternativo activado para id=1, cantidad=2. Causa: Producto no encontrado!
2026-06-09 19:59:06.154 ERROR 49597 --- [pool-2-thread-1] c.f.s.a.item.controllers.ItemController : [TimeLimiter - LATENCIA] Metodo alternativo activado para id=1, cantidad=1. Causa: timeout superado -> TimeLimiter 'itemsTL' recorded a timeout exception.
[]
```

Figura 5. Logs del microservicio servicio-items (puerto 8002): ERROR [CircuitBreaker - FALLA] para id=1, cantidad=2, y ERROR [TimeLimiter - LATENCIA] para id=1, cantidad=1 con 'timeout superado'.

3. TimeLimiter por Latencia — @TimeLimiter

3.1 Descripción

Se implementó el escenario de latencia en el endpoint **GET /ver-lento/{id}/cantidad/{cantidad}**. El método duerme 5 segundos (`TimeUnit.SECONDS.sleep(5L)`) pero el `TimeLimiter` tiene configurado un timeout de **2 segundos**. Al superarse el timeout se activa el método alternativo **metodoAlternativoLatencia()**. Este endpoint retorna `CompletableFuture<Item>` como lo requiere la anotación `@TimeLimiter`.

3.2 Código implementado

El código del método con `@TimeLimiter` se puede observar en la Figura 3 (líneas 84–97 del *ItemController.java*): el método `detalleLento()` retorna un `CompletableFuture` que ejecuta `sleep(5L)` de forma asíncrona, disparando el timeout de 2s y activando `metodoAlternativoLatencia()`.

3.3 Evidencia — Respuesta JSON del fallback (TimeLimiter)

La Figura 4 muestra la respuesta del endpoint **GET /ver-lento/1/cantidad/1**: el `TimeLimiter` corta la ejecución a los 2 segundos y retorna el fallback con marca "Sin marca (timeout)".

3.4 Evidencia — Mensajes en el logger (TimeLimiter)

La Figura 5 muestra en los logs del servicio-items el mensaje de error **[TimeLimiter - LATENCIA]** con la causa "timeout superado -> TimeLimiter 'itemsTL' recorded a timeout exception".

```
ient      : Getting all instance registry info from the eureka server
2026-06-09 19:48:20.832 INFO 46192 --- [freshExecutor-0] com.netflix.discovery.DiscoveryCl
ient      : The response status is 200
2026-06-09 19:52:50.208 INFO 46192 --- [trap-executor-0] c.n.d.s.r.aws.ConfigClusterResolv
er        : Resolving eureka endpoints via configuration
2026-06-09 19:57:50.209 INFO 46192 --- [trap-executor-0] c.n.d.s.r.aws.ConfigClusterResolv
er        : Resolving eureka endpoints via configuration
2026-06-09 19:59:09.631 INFO 46192 --- [nio-8001-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/
]         : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-06-09 19:59:09.632 INFO 46192 --- [nio-8001-exec-1] o.s.web.servlet.DispatcherServlet
         : Initializing Servlet 'dispatcherServlet'
2026-06-09 19:59:09.638 INFO 46192 --- [nio-8001-exec-1] o.s.web.servlet.DispatcherServlet
         : Completed initialization in 6 ms
```

□

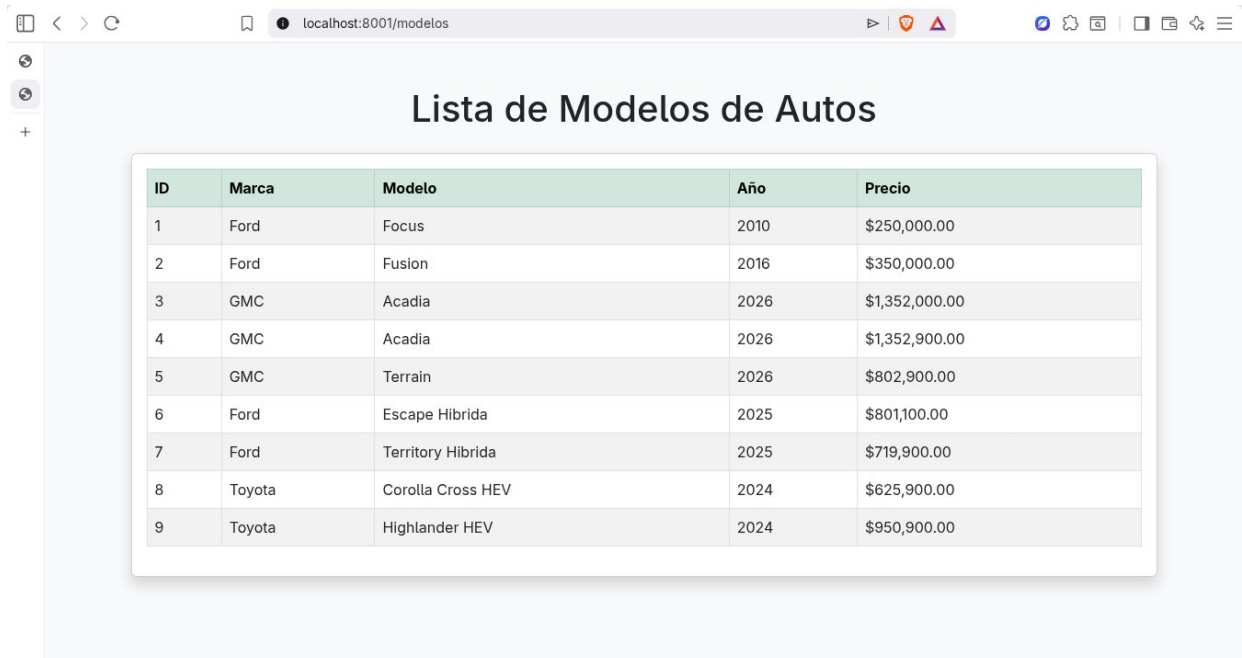
Figura 6. Logs del servidor Eureka/gateway (puerto 8001) mostrando el registro y renovación de instancias de servicio-items y servicio-productos.

4. Funcionalidad Base — GET /listar

El endpoint **GET /listar** funciona sin Circuit Breaker y representa la operación normal de consulta. Retorna la lista completa de productos registrados en la base de datos H2 del microservicio **servicio-productos**, enriquecidos con cantidad, total, IVA y número de factura calculados en **servicio-items**. La respuesta JSON es visible en la Figura 4.

5. Vista Web Estilizada — Tabla de Modelos

Como parte de la continuación de la Tarea 5, el microservicio **servicio-productos** expone una vista web en **http://localhost:8001/modelos**. Utiliza *Thymeleaf* como motor de plantillas y *Bootstrap 5* para el estilizado, mostrando los modelos de autos en una tabla responsiva con 9 registros (Ford, GMC y Toyota).



ID	Marca	Modelo	Año	Precio
1	Ford	Focus	2010	\$250,000.00
2	Ford	Fusion	2016	\$350,000.00
3	GMC	Acadia	2026	\$1,352,000.00
4	GMC	Acadia	2026	\$1,352,900.00
5	GMC	Terrain	2026	\$802,900.00
6	Ford	Escape Híbrida	2025	\$801,100.00
7	Ford	Territory Híbrida	2025	\$719,900.00
8	Toyota	Corolla Cross HEV	2024	\$625,900.00
9	Toyota	Highlander HEV	2024	\$950,900.00

Figura 7. Vista web <http://localhost:8001/modelos> — tabla Bootstrap responsiva con lista de modelos de autos (Ford Focus, Fusion; GMC Acadia, Terrain; Toyota Corolla Cross HEV, Highlander HEV).

6. Resumen de Pruebas

Se verificaron todas las funcionalidades requeridas:

Endpoint	Descripción	Respuesta esperada	Resultado
GET /listar (8002)	Lista todos los productos registrados en BD	JSON con 4+ productos, HTTP 200	✓ PASS
GET /ver/1/cantidad/2	CircuitBreaker: lanza IllegalStateException	Fallback: 'Sin marca (fallback)'	✓ PASS
GET /ver-lento/1/cantidad/1	TimeLimiter: sleep(5s) > timeout(2s)	Fallback: 'Sin marca (timeout)'	✓ PASS
GET /modelos (8001)	Vista Thymeleaf con tabla Bootstrap	Tabla HTML estilizada con 9 autos	✓ PASS

Conclusión

Se implementó exitosamente el patrón **Circuit Breaker** con Resilience4J en la arquitectura de microservicios de Spring Boot. Las cuatro actividades requeridas fueron completadas:

- **Configuración yml:** Todos los parámetros de Resilience4J fueron especificados en *application.yml* con las instancias *itemsCB* e *itemsTL* derivando de la configuración *defecto*.
- **@CircuitBreaker:** El endpoint `/ver/{id}/cantidad/{cantidad}` lanza `IllegalStateException` y activa automáticamente `metodoAlternativo()`, devolviendo datos predeterminados con marca 'Sin marca (fallback)'.
- **@TimeLimiter:** El endpoint `/ver-lento/{id}/cantidad/{cantidad}` simula 5 segundos de latencia pero el timeout de 2s activa `metodoAlternativoLatencia()` con respuesta 'Sin marca (timeout)'.
- **Logger:** Ambos métodos alternativos registran mensajes de error con prefijos **[CircuitBreaker - FALLA]** y **[TimeLimiter - LATENCIA]**, incluyendo id, cantidad y causa.